TECHNICAL ARCHITECTURE

MIRQAB Architecture

Components, data flows, deployment topologies and trust boundaries — with the actual implemented architecture clearly separated from the reference and planned architecture.

TEAM

Bdoor Fahed Alosaimi
Nouf Saleh Altalhi
Alaa Rafa Al Saad

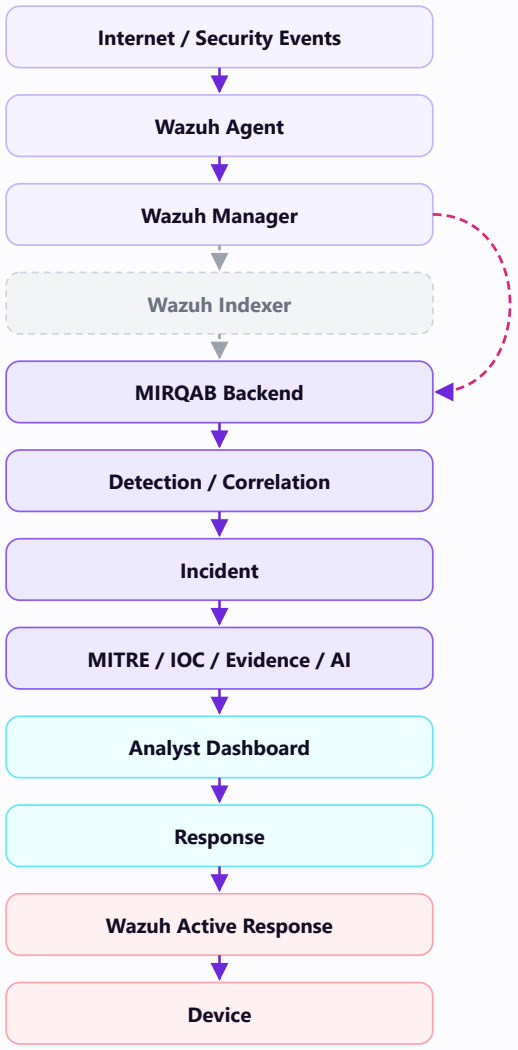
PROJECT

MIRQAB — AI-Powered Security Operations Platform for SMBs

27 September 2026

1. Reference flow vs. actual flow

The submission brief describes a reference security pipeline. The table on the right of the diagram shows how each link maps to what is **actually implemented** in MIRQAB today.

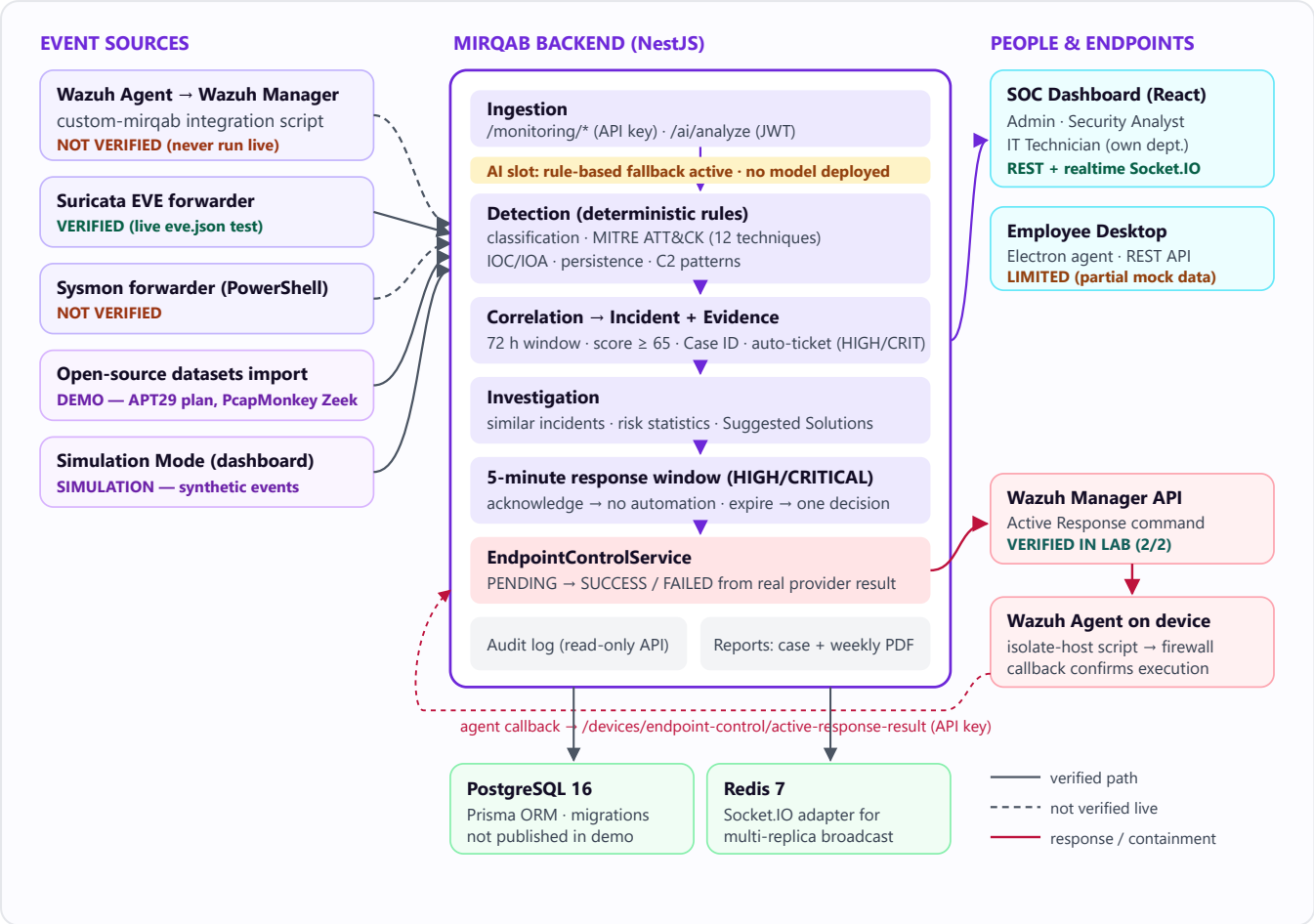


Internet / Security Events	Events from endpoints / network
Wazuh Agent	**VERIFIED IN LAB — containerized safe-test agent**
Wazuh Manager	**VERIFIED IN LAB — Manager API (Active Response)** alerts → MIRQAB via custom-mirqab script: NOT VERIFIED
Wazuh Indexer	**NOT INTEGRATED — MIRQAB never reads the Indexer**
MIRQAB Backend	**VERIFIED — /monitoring/* ingestion (Wazuh-format payloads)** also Suricata (verified), Sysmon (not verified), datasets, simulation
Detection / Correlation	**VERIFIED — deterministic rules, 72 h / score ≥ 65**
Incident	**VERIFIED — Case ID, evidence, auto-ticket, 5-min window**
MITRE / IOC / Evidence / AI	**VERIFIED — 12 MITRE techniques, IOC/IOA** AI = rule-based fallback; no model deployed (LIMITED)
Analyst Dashboard	**VERIFIED — React SOC Dashboard, realtime updates**
Response	**VERIFIED — confirmed, authorized, audited actions**
Wazuh Active Response	**VERIFIED IN LAB — 2/2, execution confirmed by callback** Wazuh GA 4.14.7: BLOCKED (lab used a release candidate)
Device	**VERIFIED IN LAB — Linux container; physical devices NOT VERIFIED** release isolation: NOT IMPLEMENTED in MIRQAB

Key difference from the reference flow

The **Wazuh Indexer is not part of MIRQAB's data path**. Alerts reach MIRQAB either from a Manager-side integration script (implemented, not verified live) or from forwarders posting directly to the verified `/monitoring/*` endpoints. Reading from the Indexer is not implemented and not claimed.

2. Component architecture (actual)

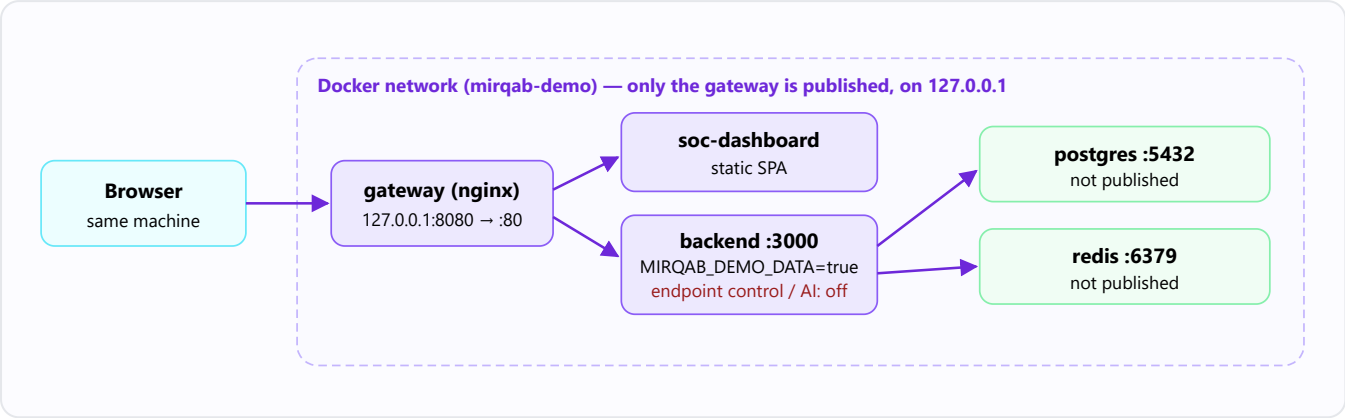


Component	Actual implementation	Planned / not implemented
Frontend	SOC Dashboard: React 18 + Vite SPA served by an nginx container. REST through Axios and React Query. Socket.IO for realtime cache invalidation. Arabic/English, RTL, dark/light. Employee Desktop: Electron 29 + React.	Full Arabic translation; packaged desktop distribution; per-employee notifications PLANNED
Backend	NestJS 10 modular monolith, `/api/v1`. Global validation, Helmet, throttling, exception filter. Scheduled jobs: the response-window sweep and the weekly report (Monday 05:00 UTC). Horizontally scalable behind nginx (`--scale backend=N`).	Dedicated security review of post-8-September features PLANNED
PostgreSQL	PostgreSQL 16 through Prisma 5 (14 migrations, applied automatically at container start). Named Docker volume. Advisory locks for once-only decisions.	Automated backups, point-in-time recovery PLANNED
Redis	Redis 7 as the Socket.IO adapter for cross-replica broadcast. Optional in single-instance mode. No primary data.	—
Socket.IO	`/realtime` namespace. JWT handshake (HS256), dashboard roles only, department/organization rooms, server-to-client only, per-IP connection limit, disconnect at token expiry.	—

Component	Actual implementation	Planned / not implemented
Nginx	Development: TLS (self-signed) on 443 with HTTP→HTTPS redirect and 5 security headers. Demo: plain HTTP on `127.0.0.1:8080`. Routes `/api/v1` and `/socket.io` to the backend, everything else to the dashboard.	CA-issued certificate on a real domain NOT DEPLOYED
Wazuh	Manager API for Active Response and optional logtest enrichment. Agent-side isolation scripts with callback confirmation. `/monitoring/wazuh` ingestion.	Indexer/Dashboard integration NOT IMPLEMENTED; Manager script live run NOT VERIFIED; GA version BLOCKED
AI layer	Provider registry with one slot. Rule-based fallbacks active. Guardrails (injection delimiting, output validation, circuit breaker, timeouts, concurrency cap). Least-privilege "agents". Native vector similarity. Statistical risk estimates.	Self-hosted model on customer hardware NOT VERIFIED NOT DEPLOYED; external AI stays development-only

3. Deployment topologies

3.1 Evaluator demo (`infra/demo` , Compose project `mirqab-demo`)



3.2 Development stack (`infra/docker`)

The same services, plus TLS on the gateway (host ports 80/443). **LIMITED**: this stack publishes PostgreSQL on `0.0.0.0:5432` and the backend on a random host port on all interfaces, for local development convenience. Do not run it on untrusted networks without binding these to `127.0.0.1`. The backend can be scaled to several replicas, and nginx re-resolves the service name so all replicas receive traffic, while Redis relays realtime events between them.

3.3 Wazuh lab (development only)

For isolation testing, the team ran the upstream `wazuh-docker` single-node stack (Manager, Indexer, Dashboard) plus a containerized safe-test agent (`IT-SRV-01`) on the development machine. This lab is **not** part of the submission package or the evaluator demo.

4. Key sequences

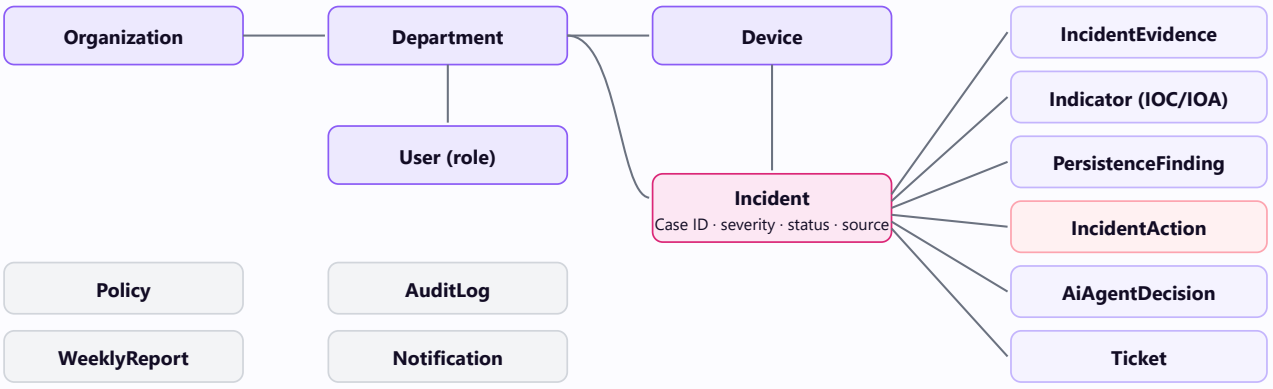
4.1 Event to incident

```
Forwarder ──POST /monitoring/wazuh (X-Api-Key)──▶ MonitoringService (normalize)
   ──▶ AiService.analyzeLog
         ├─ classify (rules | validated AI provider)      → severity, MITRE technique
         ├─ enrich   (IOC/IOA, persistence, C2, logtest)
         ├─ correlate (open cases, 72 h, score ≥ 65)       → existing case | new Case ID
         ├─ persist  incident + evidence + indicators + agent decisions
         └─ HIGH/CRITICAL → ticket + "response required" notification + window (T0+5 min)
   ──▶ EventEmitter ──▶ RealtimeGateway ──▶ scoped rooms ──▶ dashboard cache refresh
```

4.2 Five-minute window and containment

```
T0 incident (HIGH/CRITICAL) ──▶ countdown shown, notification sent
  analyst "Acknowledge"  (before T0+5)  ──▶ recorded; automated containment cancelled for good
  sweep every tick after T0+5 (no acknowledgement):
    BEGIN; advisory lock(incident); re-read state; once-only marker
      simulated | policy off | no provider ──▶ "not attempted" + reason
      otherwise ──▶ ISOLATE_DEVICE (PENDING) committed
    COMMIT ──▶ EndpointControlService ──▶ Wazuh Manager API ──▶ agent script
    agent callback ──▶ /devices/endpoint-control/active-response-result
    same action ──▶ SUCCESS | FAILED  ──▶ "containment result" notification + audit
```

5. Data model (simplified)



Every tenant-owned record carries organizationId; Incident → Department / Device relations drive department scoping.

6. Trust boundaries and controls

See the Security documentation for the full list. In summary: browser ↔ nginx (TLS in the development stack), user ↔ API (JWT + RBAC + scoping), forwarder ↔ API (API key), backend ↔ Wazuh (server-side credentials; the Manager certificate authority is trusted explicitly), agent ↔ backend (API key + device match), backend ↔ AI (server-configured endpoint, fail-closed, output validation, never able to act).

7. Actual vs. planned — summary

Area	Actual (today)	Planned
Hosting	Local Docker on the development machine	Production hosting with a real domain and CA TLS
Telemetry path	Forwarders → `/monitoring/*`	Verified Wazuh Manager integration script; broader sources
AI	Rule-based fallbacks; provider slot empty	Self-hosted model inside the customer network
Containment	Wazuh Active Response, lab-verified on Linux	Physical Windows/Linux validation; release action; GA Wazuh
Operations	Health checks, logs, manual `pg_dump`	Metrics/alerting, automated backups, DR plan, load testing